Artificial Intelligence and Deep Learning
6/1/2025

Name: K. Raman
RollNo: 23315A05..
Branch: CSE-B

Assignment - II

Unit - IV

Amy has two bags. Bag I has 7 red and 4 blue balls and bag II has 5 red and 9 blue balls. Amy draws a ball at random and it turns out to be red. Determine the probability that the ball was from the bag I.

A: This is a Baye's Theorem problem.

Given Data:

- Bag I : 7 red, 4 blue ⇒ total = 11 balls
- Bag II : 5 red, 9 blue ⇒ total = 14 balls
- Amy selects one of the bag at random → $P(\text{Bag I}) = P(\text{Bag II}) = 1/2$

- B_1 : Ball is from Bag I
- B_2 : Ball is from Bag II
- R : A red ball is drawn.

Given:

- $P(B_1) = P(B_2) = 1/2$ (since randomly chosen bag)
- Bag I has 7 red, 4 blue ⇒ $P(R|B_1) = 7/11$
- Bag II has 5 red, 9 blue ⇒ $P(R|B_2) = 5/14$

Using Bayes' Theorem

$$P(B_1|R) = \frac{P(R|B_1)\cdot P(B_1)}{P(R|B_1)\cdot P(B_1) + P(R|B_2)\cdot P(B_2)}$$

$$P(B_1|R) = \frac{7/11 \cdot 1/2}{7/11 \cdot 1/2 + 5/14 \cdot 1/2} = \frac{7/22}{7/22 + 5/28}$$

$$P(B_1|R) = \frac{98/308}{98/308 + 55/308} = \frac{98}{153}$$

$$\therefore P(B_1|R) = 98/153$$

2. Explain Policy-iteration Algorithm?

Ans. Policy Iteration is a method used to help a computer or agent learn how to make the best decisions step by step in a certain environment (like a game or robot navigation). It works with something called a Markov Decision Process (MDP)

Key Terms

- Policy (π): A mapping from states to actions.
- Value Function ($V\pi$): Expected return when following policy π from a state.
- Bellman Equation: Used to calculate the value functio

Steps in Policy Iteration:

It involves two main steps repeated iteratively:

Step 1: Policy Evaluation (Check how good your guess is)

- You follow your guessed path.
- You calculate how good or bad each location (state) is when following this path.

Step 2: Policy Improvement (Make a better guess)

- Now that you know which places are good or bad,
- You change your path to choose better actions from each place.

Repeat:

- Keep doing these two steps until your path doesn't change anymore.
- When that happens, you've found the best path (optimal policy)

- Policy Iteration = Keep checking and improving your decisions until they can't get any better.

3. ~~write~~ shotes to

3. Write short note on Partially Observable MDP?

Ans: A Partially Observable MDP (POMDP) is an extension of an MDP where the agent cannot directly observe the current state of the environment

Key Components:

- States (S): Actual states of the environment.
- Actions (A): Choices available to the agent.
- Observations (O): What the agent can observe (incomplete info about the state)
- Transition Model (P): $P(s'|s,a)$ - probability of reaching state s' from s using action a.
- Observation Model: $P(o|s',a)$ - probability of observation o after action a leads to state s'.
- Reward function (R): Expected reward for taking action in a state.
- Belief State: A probability distribution over all possible states, representing the agent's guess of the current state.

Agent's Goal: To choose actions that maximize expected reward over time, based on belief states, not actual states.

Use cases:

- Robotics (uncertain sensors)
- Medical diagnosis
- Navigation with noisy GPS!

POMDPs model decision-making under uncertainty and partial observability, using belief states to make informed decisions.

Unit - V

1. Write short notes on Max-Pooling in CNN?

Ans: Max-Pooling is a down sampling operation used in Convolutional Neural Networks (CNNs) to reduce the spatial dimension (width * height) of feature maps.

Key Points:

- Purpose:
  - Reduce computation
  - Control over-fitting
  - Retain important feature
- How it works.
  - A window (e.g. 2x2) slides over the input feature map.
  - For each window, it output the maximum value
- Example: Input

  $\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \Rightarrow$ Max-Pooling (2 X 02) $\Rightarrow 4$
- Common settings:

  Pool size: 2x2.

  Stride: 2

Max-Pooling simplifies feature maps by keeping the most important values, making the network more efficient and robust to small translations.

2. Write short notes on role of neurons in human vision?

Ans:- Neurons are special cells in our brain and eyes that can help us see the world.

- They carry signals from the eyes to the brain so we can understand what we are looking at.
- How vision works with neurons:

1. Light enters the eye and hits the retina (back part of the eye).
2. The retina has special neurons called photoreceptors:
   - Rods: helps us see in dim light (black & white)
   - Cones: help us see colors.

3. [illegible] send signals to other neurons in
4. the eye.
4. Signals go to the optic nerve, which carries them to the visual cortex in the brain.
5. The brain neurons process the image - like shape, color, movement - and help us understand what we see.

- Important Roles of Neurons:
- Detect light and color
- Recognize shapes and objects
- Track motion
- Send fast messages from the eye to the brain
- Help the brain form visual memory and perception
- Neurons act like messengers that turn light into signals and help our brain understand what we see. Without them, we can't recognize the world around us.

3. Design an AND gate using a Sample Neural Network?

Ans: Designing an AND gate using a Sample Neural Network.

- We'll create a simple neural network that acts like an AND logic gate using:
- ~~2 Inputs~~: 2 Inputs: A and B
- ~~1 Neuron~~: 1 neuron (no hidden layer)
- Activation Function: Step function or Sigmoid
- Weights and Bias: Manually set to mimic AND logic.

Truth Table for AND Gate

Input A	Input B	Output
0	0	0
0	1	0
1	0	0
1	1	1

- Neural Network Equation:
- Output = Activation $(w_1 \cdot A + w_2 \cdot B + b)$
- Choose Parameter (weight and bias)
  - $w_1 = 1$
  - $w_2 = 1$
  - $b = -1.5$
  - Activation: step function

$$step(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0. \end{cases}$$

- How it works

A	B	A+B-1.5	Output
0	0	0-1.5 = -1.5	0
0	1	1-1.5 = -0.5	0
1	0	1-1.5 = -0.5	0
1	1	2-1.5 = 0.5	1

The neuron activates only when both inputs are 1. Just like an AND gate.

- A single neuron can mimic an AND gate using proper weight and bias
- It learns or mimics the rule: "Only give if both inputs 1."

Unit-VI

1. Write short notes on Pole-Cart problem?

Ans: The Pole-Cart Problem, also called the Inverted Pendulum problem, is a classic example in Reinforcement Learning (RL) and Control System.

- The setup.
  - A cart can move left or right on a track.
  - A pole is attached to the cart and stands upright.
  - The goal is to keep the pole balanced and prevent it from falling.

- What does the agent do?
- The agent decides whether to push the cart left or right at every step.
- It learns to make these decisions using trial and error.
- Why it's used:
- It's a benchmark problem to test:
  - Control algorithms
  - Reinforcement learning methods
  - Decision-making in real time
- Key Concepts Learned:
- Balancing unstable systems
- Feedback control
- Continous state space
- Application of Q-learning and Deep Q Networks (DQN)
- Real-life uses:
- Robotics (balance and stability)
- Self-balancing tech, Vehicels (like segways)
- Autonomous systems.
- The pole cart problem teaches how to make smart decision to keep an unstable system balanced by learning through rewards and actions.

2. Write short notes on Deep Q-Networks).

Ans: A Deep Q-network (DQN) is a type of Reinforcement learning ~~algorit~~ algorithm that combines:

- Q-Learning (a popular RL method)
- With a Deep Neural Network (for handling complex input like images or big data)

- Why use it?
- Traditional Q-learning stores values in Q-table, which becomes too large for big environments
- DQN replaces the Q-table with a neural network, allowing it to learn in complex environments (like games, robotics, etc).

Components of DQN:

1. Neural Network: Takes state as input and outputs Q-values for all possible actions.
2. Replay Buffer: Stores past experiences (state, action, reward, next state) to train the network more efficiently.
3. Target Network: A second network used to stabilize learning.
4. Exploration Vs. Exploitation: Uses an ε-greedy strategy to balance trying new actions vs. using the best known ones.

- Example Use Case:
  - Atari Games: DQN was famously used by DeepMind to play and beat human scores in classic Atari games using just pixel input.
  - A DQN is like a smart brain that learns how to take the best actions in a game or environment by using deep learning instead of a lookup table.

3. Write short notes on deep recurrent Q Networks?

Ans: A Deep Recurrent Q-Network (DRQN) is an advanced version of a Deep Q-Network (DQN) that includes a recurrent neural network (RNN) - usually an LSTM (Long Short-Term Memory) layer.

- Why do we need it?

Sometimes, the agent cannot see the full environment (like in games with fog or memory-based decisions). In such cases, the agent must remember past observations to make better decisions.

- Key Difference from DQN:
  - DQN: Takes a single frame (snapshot) as input.
  - DRQN: Takes a sequence of frames and uses memory (via RNN) to understand what's happening over time.

• Structure of DRQN:

1. Input: A sequence of past observations (not just one)
2. LSTM layer: stores memory of previous steps
3. Fully connected layer: Output Q-values (like in DQN)

• Use cases:

- Partially observable environments.
- ~~LSTM in~~
- Robotics with sensors
- Games where the full state is not visible

• A DRQN adds memory to a DQN, so the agent can make smart decisions even when it doesn't have complete information at each moment.